

# Graph Algorithms

- [Definition of Graphs](#)
- [Graph Traversal](#)
  - [Intro](#)
  - [DFS \(Depth First Search\)](#)
  - [BFS \(Breadth First Search\)](#)
- [Application of DAG](#)
  - [Intro](#)
  - [Topological Sort](#)
  - [Critical Path](#)
- [Shortest Path Problem](#)
  - [Things you should know:](#)
  - [Floyd-Warshall](#)
  - [Bellman-Ford](#)
  - [Dijkstra](#)
- [Network Flow](#)
  - [Intro](#)
  - [Ford-Fulkerson](#)

⚠ 以下所有代码均由邻接矩阵实现。

⚠ 所有知识点均来自刘圣鑫老师课件中。想要更加系统性的知识结构可以 check [OI Wiki](#)

## Definition of Graphs

😊 Get the background knowledge at "[GraphTheory](#)"(not finished yet).

```
typedef char VexType;
typedef float AdjType;

typedef struct {
    vextype vexts[n]; // Vertices
    adjtype arcs[n][n]; // Adjacent Matrix
} Graph;
```

| Method           | Function |
|------------------|----------|
| CreatGraph(&G)   | 建立图      |
| DestroyGraph(&G) | 销毁图      |
| InsertVEx(&G, v) | 插入顶点     |

| Method                     | Function |
|----------------------------|----------|
| DeleteVEx(&G, v)           | 删除顶点     |
| InsertArc(&G, v, w)        | 插入弧      |
| DeleteArc(&G, v, w)        | 删除弧      |
| DFSTraVerse(G, v, Visit()) | 深度优先搜索   |
| BFSTraVerse(G, v, Visit()) | 广度优先搜索   |

# Graph Traversal

## Intro

- 当无向图为非连通图时，从图中某一顶点出发，利用深度优先搜索算法或广度优先搜索算法不可能遍历到图中的所有顶点，只能访问到该顶点所在的极大连通子图（连通分量）的所有顶点。
- 若从无向图的每一个连通分量中的一个顶点出发进行遍历，可求得无向图的所有连通分量。
- 树的先根遍历是一种深度优先搜索策略，树的层次遍历是一种广度优先搜索策略。

## DFS (Depth First Search)

### 📖 算法思想

- (1) 首先访问图中某一个顶点  $V_i$ ，以该顶点为出发点；
- (2) 任选一个与顶点  $V_i$  邻接的未被访问的顶点  $V_j$  进行访问；
- (3) 以  $V_j$  为新的出发点递归地继续进行深度优先搜索，直至图中所有和  $V_i$  有路径的顶点均被访问到。

从图中的某些顶点出发，使用 DFS 搜索长度为  $k$  的路径（不重复顶点），并输出所有可能的路径。

■ 常用模版如下：

```
int adj[MAX_N][MAX_N]; // 邻接矩阵
int book[MAX_N];       // 标记是否访问过
int path[MAX_N];       // 当前路径
int n, m, k;           // n个顶点，m条边，要找长度为k的路径
int ans = 0;

// cur 表示当前路径长度，v 是当前所在的顶点
void DFS(int v, int cur) {
    path[cur] = v;
    book[v] = 1;
```

```

    if (cur == k) {
        // 到达所需长度，输出路径
        for (int i = 0; i <= k; i++) {
            printf("%d ", path[i]);
        }
        printf("\n");
        ans++;
        book[v] = 0; // 回溯前取消标记
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (adj[v][i] && !book[i]) {
            DFS(i, cur + 1);
        }
    }

    book[v] = 0; // 回溯
}

int main() {
    int u, v;
    printf("请输入顶点数 n，边数 m，路径长度 k：\n");
    scanf("%d %d %d", &n, &m, &k);

    // 初始化邻接矩阵
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            adj[i][j] = 0;

    printf("请输入 %d 条边（格式：起点 终点）：\n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // 若为无向图
    }

    printf("所有长度为 %d 的路径如下（不重复顶点）：\n", k);
    for (int i = 1; i <= n; i++) {
        DFS(i, 0); // 从每个顶点出发
    }

    printf("共找到 %d 条路径。", ans);
    return 0;
}

```

对于非连通图的 DFS：

(1) 首先将图中每个顶点的访问标志设为FALSE，之后搜索图中每个顶点：

- (2) 若已被访问过，则该顶点一定是落在图中已求得的连通分量上；  
(3) 若还未被访问，则从该顶点出发遍历图，可求得图的另一个连通分量。

模版程序如下：

```
int adj[MAX_VERTICES][MAX_VERTICES]; // 邻接矩阵
int visited[MAX_VERTICES];           // 标记数组
int n;                               // 顶点个数

void DFS(int v) {
    visited[v] = 1;
    printf("%d ", v); // 打印当前访问的顶点

    for (int i = 0; i < n; i++) {
        if (adj[v][i] && !visited[i]) {
            DFS(i);
        }
    }
}

void DFSTraversal() {
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            printf("Connected component starting from vertex %d: ", i);
            DFS(i);
            printf("\n");
        }
    }
}

// 调用示例
int main() {
    int edges, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // 初始化邻接矩阵
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    // 读入边
```

```

printf("Enter edges (format: u v):\n");
for (int i = 0; i < edges; i++) {
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    adj[v][u] = 1; // 无向图
}

printf("DFS traversal of the graph:\n");
DFSTraversal();

return 0;
}

```

## BFS (Breadth First Search)

所谓宽度优先，就是每次都尝试访问同一层的节点。如果同一层都访问完了，再访问下一层。

这样做的结果是，BFS 算法找到的路径是从起点开始的最短合法路径。换言之，这条路径所包含的边数最小。

 算法思想：

- (1) 首先访问图中某一个指定的出发点  $v_i$ ;
- (2) 然后依次访问  $v_i$  的所有邻接点  $v_{i1}, v_{i2}, \dots, v_{it}$ ;
- (3) 再依次以  $v_{i1}, v_{i2}, \dots, v_{it}$  为顶点，访问各顶点未被访问的邻接点，依此类推，直到图中所有顶点均被访问为止。
- (4) 若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中所有顶点都被访问到为止。

 模版程序如下：

```

int adj[MAX_N][MAX_N]; // 邻接矩阵
int visited[MAX_N];     // 访问标记数组
int queue[MAX_N];       // 队列
int front = 0, rear = 0;

int n, m; // n: 顶点数, m: 边数

void BFS(int start) {
    front = rear = 0; // 初始化队列
    queue[rear++] = start; // 起点入队
    visited[start] = 1; // 标记为已访问

    while (front < rear) {
        int cur = queue[front++]; // 出队
        printf("%d ", cur); // 访问当前顶点
    }
}

```

```

        // 遍历所有邻接点
        for (int i = 1; i <= n; i++) {
            if (adj[cur][i] && !visited[i]) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }
}

int main() {
    int u, v, start;

    printf("请输入顶点数 n 和边数 m: \n");
    scanf("%d %d", &n, &m);

    // 初始化邻接矩阵
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            adj[i][j] = 0;

    printf("请输入 %d 条边（格式：起点 终点）: \n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // 如果是无向图
    }

    printf("请输入起点编号: \n");
    scanf("%d", &start);

    printf("BFS访问顺序: \n");
    BFS(start);
    printf("\n");

    return 0;
}

```

## Application of DAG

### Intro

一个无环的有向图称作有向无环图 (directed acyclic graph, DAG)。

DAG 在工程计划和管理方面应用广泛。几乎所有的工程都可分为若干个称作“活动”的子工程，并且这些子工程之间通常受着一定条件的约束，即某些子工程必须在另外的一些子工程

完成之后才能开始。

对整个工程和系统，人们主要关心的是两方面的问题：

- 工程能否顺利进行.  $\rightarrow$  **Topological Sort** (拓扑排序)
- 完成整个工程所必须的最短时间.  $\rightarrow$  **Critical Path** (关键路径)

---

## Topological Sort

在一个 DAG 中，我们将图中的顶点以线性方式进行排序，使得对于任何的顶点  $u$  到  $v$  的有向边  $\langle u, v \rangle$ ，都可以有  $u$  在  $v$  的前面。

在以上工程背景的叙述中可表示为：

给定一个 DAG，如果从  $i$  到  $j$  有边，则认为  $j$  依赖于  $i$ 。如果  $i$  到  $j$  有路径 (Path)，则称  $j$  间接依赖于  $i$ 。拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。

由此所得顶点的线性序列称之为拓扑有序序列。

 算法思想：

- (1) 在有向图中选一个入度为 0 的顶点输出。
- (2) 从图中删除该顶点及所有它的出边。
- (3) 重复执行 (1) 和 (2)，直到全部顶点均已输出，或图中剩余顶点的入度均不为 0 (说明图中存在回路，无法继续拓扑排序)。

设 DAG 有  $n$  个顶点， $e$  条边。则对该 DAG 进行拓扑排序的复杂度为  $O(n + e)$ ，分析如下：

- (1) 对  $e$  条有向边求各顶点的入度的时间复杂度是  $O(e)$ ；
- (2) 初始建立入度为 0 的顶点栈，要检查所有顶点一次，执行时间为  $O(n)$ ；
- (3) 排序中，每个顶点入、出栈各一次，每个边表结点被检查一次，执行时间为  $O(n + e)$ ；

 模版程序如下：

```
int adj[MAX_N][MAX_N]; // 邻接矩阵
int indegree[MAX_N];    // 入度数组
int stack[MAX_N];        // 栈
int top = -1;

int n, m; // 顶点数和边数

void topo_sort() {
    int count = 0;

    // 将所有入度为 0 的顶点入栈
    for (int i = 1; i <= n; i++) {
        if (indegree[i] == 0) {
            stack[++top] = i;
        }
    }
}
```

```

    }
}

while (top != -1) {
    int vi = stack[top--]; // 弹出栈顶元素
    printf("%d ", vi);    // 输出顶点
    count++;

    // 遍历 vi 的出边
    for (int j = 1; j <= n; j++) {
        if (adj[vi][j]) {
            indegree[j]--; // 终点入度减 1
            if (indegree[j] == 0) {
                stack[++top] = j; // 入度为 0 入栈
            }
        }
    }
}

printf("\n");

if (count < n) {
    printf("图中存在回路，无法进行拓扑排序。\\n");
} else {
    printf("拓扑排序成功。\\n");
}
}

int main() {
    int u, v;

    printf("请输入顶点数 n 和边数 m:\\n");
    scanf("%d %d", &n, &m);

    // 初始化邻接矩阵和入度数组
    for (int i = 1; i <= n; i++) {
        indegree[i] = 0;
        for (int j = 1; j <= n; j++) {
            adj[i][j] = 0;
        }
    }

    printf("请输入 %d 条有向边（格式：起点 终点）:\\n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        if (!adj[u][v]) { // 避免重复边影响入度
            adj[u][v] = 1;
            indegree[v]++;
        }
    }
}

```



```
printf("拓扑排序结果:\n");
topo_sort();

return 0;
}
```

## Critical Path

AOE (Activity On Edge) 网：带权的有向图无环图，顶点表示事件，边表示活动，权表示活动持续的时间。

特点：有且只有一个源点 (表示项目开始)；有且只有一个汇点 (表示项目结束)。

关键路径：在 AOE 网中，从源点到汇点的路径长度最长的路径，即决定整个项目工期的路径。

 算法：

(1) 拓扑排序：用于确定事件之间的先后关系。

(2) 求事件的最早发生时间  $ve[i]$

从源点出发正向计算： $ve[j] = \max(ve[i] + dut[i, j])$

(3) 求事件的最迟发生时间  $vl[i]$

从汇点出发反向计算： $vl[i] = \min(vl[j] - dut[i, j])$

(4) 计算每条活动的最早开始时间  $e[i]$  和最迟开始时间  $l[i]$

对于活动  $(j, k)$ ： $e[i] = ve[j]$ ,  $l[i] = vl[k] - dut[j, k]$

(5) 判断关键活动

若某活动满足： $e[i] = l[i]$ ，则该活动为关键活动。

在拓扑排序求  $ve[i]$  和逆拓扑有序求  $vl[i]$  时，所需时间为  $O(n + e)$ ；

求各个活动的  $e[k]$  和  $l[k]$  时所需时间为  $O(e)$ ；

总共花费时间仍然是  $O(n + e)$ 。

代码如下：

```
typedef struct Edge {
    int to;
    int weight;
    int next;
} Edge;

Edge edges[MAXE];
int head[MAXN], indegree[MAXN], outdegree[MAXN];
int ve[MAXN], vl[MAXN];
int stack[MAXN], top;
int n, m, edge_cnt = 0;
```

```

void add_edge(int u, int v, int w) {
    edges[edge_cnt].to = v;
    edges[edge_cnt].weight = w;
    edges[edge_cnt].next = head[u];
    head[u] = edge_cnt++;
    indegree[v]++;
    outdegree[u]++;
}

// 拓扑排序，计算 ve[]
int topo_sort() {
    int temp_stack[MAXN], temp_top = -1;
    top = -1;
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            temp_stack[++temp_top] = i;
        }
        ve[i] = 0; // 初始化
    }

    int count = 0;

    while (temp_top != -1) {
        int u = temp_stack[temp_top--];
        stack[++top] = u; // 记录拓扑排序序列
        count++;

        for (int i = head[u]; i != -1; i = edges[i].next) {
            int v = edges[i].to;
            if (--indegree[v] == 0) {
                temp_stack[++temp_top] = v;
            }
            if (ve[u] + edges[i].weight > ve[v]) {
                ve[v] = ve[u] + edges[i].weight;
            }
        }
    }

    return count == n; // 如果不是 DAG，返回 0
}

// 计算 vl[]
void compute_vl() {
    for (int i = 0; i < n; i++) {
        vl[i] = ve[n - 1]; // 初始化为最大 ve
    }

    for (int i = top; i >= 0; i--) {
        int u = stack[i];
        for (int j = head[u]; j != -1; j = edges[j].next) {

```

```

        int v = edges[j].to;
        if (vl[v] - edges[j].weight < vl[u]) {
            vl[u] = vl[v] - edges[j].weight;
        }
    }
}

// 输出关键路径
void critical_path() {
    for (int u = 0; u < n; u++) {
        for (int i = head[u]; i != -1; i = edges[i].next) {
            int v = edges[i].to;
            int dut = edges[i].weight;
            int e = ve[u];
            int l = vl[v] - dut;
            if (e == l) {
                printf("关键活动: (%d -> %d), 持续时间: %d\n", u, v, dut);
            }
        }
    }
}

int main() {
    int u, v, w;
    printf("请输入事件数 n 和活动数 m: \n");
    scanf("%d %d", &n, &m);

    // 初始化
    edge_cnt = 0;
    memset(head, -1, sizeof(head));
    memset(indegree, 0, sizeof(indegree));
    memset(outdegree, 0, sizeof(outdegree));

    printf("请输入 %d 条有向边(起点 终点 持续时间) : \n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &u, &v, &w);
        add_edge(u, v, w);
    }

    if (!topo_sort()) {
        printf("图中存在环, 无法计算关键路径! \n");
        return 0;
    }

    compute_vl();

    printf("\n关键路径如下: \n");
    critical_path();
}

```

```
printf("项目最短完成时间为 : %d\n", ve[n - 1]);

return 0;
}
```

## Shortest Path Problem

⚠ 最短路径存在的必要条件为图中不能存在负环。

### Things you should know:

- 对于图  $G = (V, E)$ ，记  $|V| = n, |E| = m$ ;
- $w(u, v)$  为  $(u, v)$  这一条边的边权。
- 设  $s$  为最短路的源点， $D(u)$  为  $s$  点到  $u$  点的实际最短路径长度， $dis(u)$  为  $s$  点到  $u$  点的估计最短路径长度。任何时候都有  $dis(u) \geq D(u)$ 。特别地，当最短路算法终止时，应有  $dis(u) = D(u)$  成立。
- 对于边权为正的图：任意两个结点之间的最短路不会经过重复的结点和边；任意两个结点之间的任意一条最短路结点数不会超过  $n$ ，边数不会超过  $n - 1$ 。

## Floyd-Warshall

适用于任何存在最短路径的图，可求图中任意两个结点之间的最短路径。  
复杂度较高，因此适用于规模较小的图。

我们定义一个三维数组  $f[k][x][y]$ ，表示结点  $x$  到结点  $y$  在点导出子图  $G[V']$  中的最短路径长度，其中  $V' = \{V_1, V_2, \dots, V_k\}$ 。也就是说该最短路径只允许经过  $\{V_1, V_2, \dots, V_k\}$ ，注意这里的  $x, y$  不一定属于  $V'$ 。

很显然， $f[n][x][y]$  就是结点  $x$  到结点  $y$  的最短路径长度（因为  $V' = 1, 2, \dots, n$  即为  $V$  本身，其表示的最短路径就是所求路径）。

接下来我们只需要考虑如何求出  $f$  数组的值即可。我们将  $\{V_1, V_2, \dots, V_k\}$  简记为  $\{1, 2, \dots, k\}$ 。

- $f[0][x][y]$ ：当  $x$  与  $y$  间有直接相连的边时，为它们的边权；当  $x = y$  时为 0；否则为  $+\infty$ 。
- $f[k][x][y] = \min(f[k-1][x][y], f[k-1][x][k] + f[k-1][k][y])$ ：  $f[k-1][x][y]$  为不经过  $k$  点的最短路径，而  $f[k-1][x][k] + f[k-1][k][y]$  为经过  $k$  节点的最短路径。

不难理解该算法时间复杂度是  $O(n^3)$ ：我们需要依次增加问题规模 ( $k$  从 1 到  $n$ )，判断任意两点在当前问题规模下的最短路。

而由于这里使用了三维数组，空间复杂度也自然是  $O(n^3)$ 。

看到这里你以为完了吗？其实我们可以证明该数组的第一维完全是可以省略的：

对于给定的  $k$ ，当根据前面的递推式更新  $f[k][x][y]$  时，涉及的元素总是来自  $f[k-1]$  数组的第  $k$  行和第  $k$  列。

我们不难发现，

```
f[k][k][y] = min(f[k-1][k][y], f[k-1][k][k]+f[k-1][k][y])
             = min(f[k-1][k][y], f[k-1][k][y])
             = f[k-1][k][y]
```

对于  $f[k][x][k]$  的证明类似。

因此对于给定的  $k$ ，当更新  $f[k][k][y]$  或  $f[k][x][k]$ ，总是不会发生数值更新。也就是说如果省略第一维，在给定的  $k$  下，每个元素的更新中使用到的元素都没有在这次迭代中更新，因此第一维的省略并不会影响结果。

于是，我们的递推式可以简化为： $f[x][y] = \min(f[x][y], f[x][k] + f[k][y])$ 。

我们将空间复杂度优化到了  $O(n^2)$ ，**beautiful!**

示例代码如下：

```
int f[MAXN][MAXN];
int n, m;

int main() {
    int u, v, w;

    printf("请输入顶点数 n 和边数 m: \n");
    scanf("%d %d", &n, &m);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j)
                f[i][j] = 0;
            else
                f[i][j] = INF;
        }
    }

    printf("请输入 %d 条边 (格式: 起点 终点 权值) : \n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &u, &v, &w);
        if (f[u][v] > w)
            f[u][v] = w; // 选择最小边权
    }
}
```

```

// 主算法：floyd 简化版，三重循环，空间复杂度  $O(n^2)$ 
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (f[i][k] < INF && f[k][j] < INF) {
                if (f[i][j] > f[i][k] + f[k][j]) {
                    f[i][j] = f[i][k] + f[k][j];
                }
            }
        }
    }
}

// 输出最终最短路径结果，略

return 0;
}

```

## Bellman-Ford

Bellman-Ford 算法是一种基于松弛 (relax) 操作的最短路算法，可以求出有负权的图的最短路，并可以对最短路不存在的情况进行判断。

 算法思想：

对于边  $(u, v)$ ，松弛操作的定义为：

$$\text{dis}(v) = \min\{\text{dis}(v), \text{dis}(u) + w(u, v)\}$$

即我们尝试通过路径  $S \rightarrow u \rightarrow v$ （其中  $S \rightarrow u$  是最短路径）来更新  $v$  的最短路径长度。若该路径更优，则更新。

不断尝试对图中所有边进行松弛，当某一轮循环中未发生任何成功的松弛操作时，算法停止。

 正确性证明：

**Bellman-Ford** 算法的核心思想为对于长度为  $k$  的最短路径，对所有边以任意顺序进行松弛  $k$  轮即可搜索得到该最短路径。

我们使用归纳法证明以上结论。

设从起点  $s$  到达顶点  $t$  的最短路径为  $s \rightarrow v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_{k-1} \rightarrow v_k \rightarrow t$ 。

当  $k = 0$  时，该结论显然成立；

经过  $k - 1$  轮松弛之后，所有长度为  $k - 1$  的最短路径均已经被搜索得到，即  $\text{dis}(v_{k-1})$  即为  $s$  到  $v_{k-1}$  的最短路径。

在进行第  $k$  轮松弛时， $\text{dis}(v_k)$  的值会被边  $v_{k-1} \rightarrow v_k$  松弛，因此根据松弛操作的定义式，我们可以得到  $\text{dis}(v_k) \leq \text{dis}(v_{k-1}) + w(v_{k-1}, v_k)$ 。又因为  $\text{dis}(v_k)$  肯定不会小于最短路径的长度

$dis(v_k) \geq dis(v_{k-1}) + w(v_{k-1}, v_k)$ ，因此  $dis(v_k) = dis(v_{k-1}) + w(v_{k-1}, v_k)$ 。则在经过  $k$  轮松弛后，长度为  $k$  的最短路径被搜索到的。

由于起点  $s$  无法到达图中的任一负环，因此起点  $s$  到其他所有顶点的最短路径不包括环，因此起点  $s$  到其他顶点的最短路径最多包括  $|V| - 1$  条边。也就是说最多经过  $V - 1$  轮松弛，顶点  $s$  到其余顶点的最短路径便可搜索得到，这也就是为什么循环的次数为  $|V| - 1$ 。

#### ✅ 复杂度分析：

- 每轮循环对所有  $m$  条边进行松弛操作，时间复杂度为  $O(m)$
- 最多循环  $n - 1$  轮 ( $n$  为顶点数)，因为最短路径的边数最多为  $n - 1$
- 总时间复杂度：  $O(nm)$

若图中存在从源点  $S$  可达的负环，松弛操作会无限进行。但根据上述分析：

- 对于正常情况（存在最短路径），松弛操作最多执行  $n - 1$  轮
- 若第  $n$  轮仍可松弛，说明图中存在从  $S$  出发可达的负环

需要注意的是，以某一点为源点跑 **Bellman-Ford** 算法时，如果没有给出存在负环的结果，只能说明从该点出发不能抵达一个负环，而不能说明图上不存在负环。

因此如果需要严格地判断整个图上是否存在负环，最严谨的做法是建立一个超级源点，向图上每个节点连一条权值为  $0$  的边，然后以超级源点为起点执行 **Bellman-Ford** 算法。

Naive 的实现如下：

```
typedef struct {
    int u, v;    // 边的起点和终点
    int w;       // 边的权值
} Edge;

Edge edges[MAXM];    // 边集合
int dis[MAXN];       // 存储源点到每个点的最短距离
int n, m;            // 顶点数、边数
int source;          // 源点编号（从1开始）

int BellmanFord(int source) {
    // 初始化距离
    for (int i = 1; i <= n; i++) {
        dis[i] = INF;
    }
    dis[source] = 0;

    // 最多进行 n-1 轮松弛操作
    for (int i = 1; i <= n - 1; i++) {
        int updated = 0;
        for (int j = 0; j < m; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;
```

```

        if (dis[u] != INF && dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            updated = 1;
        }
    }
    if (!updated) break; // 无更新可提前终止
}

// 第 n 轮：检测负权环
for (int j = 0; j < m; j++) {
    int u = edges[j].u;
    int v = edges[j].v;
    int w = edges[j].w;
    if (dis[u] != INF && dis[v] > dis[u] + w) {
        return 0; // 存在负环
    }
}

return 1; // 无负环，返回成功
}

int main() {
    printf("请输入顶点数 n 和边数 m: \n");
    scanf("%d %d", &n, &m);

    printf("请输入 %d 条边 (格式: 起点 终点 权值): \n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    printf("请输入源点编号: \n");
    scanf("%d", &source);

    if (BellmanFord(source)) {
        printf("源点 %d 到各点的最短路径为: \n", source);
        for (int i = 1; i <= n; i++) {
            if (dis[i] == INF)
                printf("点 %d 不可达\n", i);
            else
                printf("点 %d : %d\n", i, dis[i]);
        }
    } else {
        printf("图中存在负权环，无法求解最短路径。 \n");
    }

    return 0;
}

```

Bellman-Ford 可以使用队列、栈等数据结构进行优化，具体可以 check [this](#)



# Dijkstra

一种求解非负权图上单源最短路径的算法。

 算法思想：

将结点分成两个集合 (类比插入排序)：已确定最短路径长度的点集 (记为  $S$  集合) 的和未确定最短路径长度的点集 (记为  $T$  集合)。一开始所有的点都属于  $T$  集合。

假设从点  $s$  出发，我们初始化  $dis(s) = 0$ ，其他点的  $dis$  为  $+\infty$ 。

重复以下操作直到  $T$  集合为空：

1. 若为初始状态，则将  $s$  移到  $S$  集合中；否则，从  $T$  集合中选取离当前节点距离最小 (有限) 的结点，移到  $S$  集合中。
2. 对刚刚被加入  $S$  集合的结点的所有出边执行松弛操作。

 正确性证明：

简单来说，我们要证明的就是在执行 1 操作时，取出的结点  $u$  的最短路径已经被确定，即满足  $D(u) = dis(u)$ 。我们使用归纳法进行证明。

初始时  $S = \emptyset$ ，结论成立。

接下来用反证法。设  $u$  点为算法中第一个在加入  $S$  集合时不满足  $D(u) = dis(u)$  的点。

由于  $s$  点一定满足  $D(s) = dis(s) = 0$ ，且它一定是第一个加入  $S$  集合的点，因此将  $u$  加入  $S$  集合前， $S \neq \emptyset$ 。

如果不存在  $s$  到  $u$  的路径，则  $D(u) = dis(u) = +\infty$ ，与假设矛盾。于是一定存在  $s$  到  $u$  的路径  $s \rightarrow x \rightarrow y \rightarrow u$ ，其中  $y$  为  $s \rightarrow u$  路径上第一个属于  $T$  集合的点，而  $x$  为  $y$  的前驱结点 (显然  $x \in S$ )。需要注意的是，可能存在  $s = x$  或  $y = u$  的情况，即  $s \rightarrow x$  或  $y \rightarrow u$  可能是空路径。

根据归纳假设，在  $u$  结点之前加入的结点都满足  $D(u) = dis(u)$ ，所以在  $x$  点加入到  $S$  集合时，有  $D(x) = dis(x)$ 。此时边  $(x, y)$  会被松弛，从而可以证明，将  $u$  加入到  $S$  时，一定已经有  $D(y) = dis(y)$  成立。

下面证明  $D(u) = dis(u)$  成立。

在路径  $s \rightarrow x \rightarrow y \rightarrow u$  中，因为图上所有边的权值非负，因此  $D(y) \leq D(u)$ ，从而可以做如下推导  $dis(y) = D(y) \leq D(u) \leq dis(u)$ 。

但是因为  $u$  结点在 1 过程中被取出  $T$  集合时， $y$  结点还没有被取出  $T$  集合，因此此时有  $dis(u) \leq dis(y)$ 。

从而得到  $dis(y) = D(y) = D(u) = dis(u)$ ，这与  $D(u) \neq dis(u)$  的假设矛盾，故假设不成立。

因此我们证明了，1 操作每次取出的点，其最短路径已经被确定。命题得证。

注意到证明过程中的关键不等式  $D(y) \leq D(u)$  是在图上所有边的权值非负的情况下得出的。当图上存在负权边时，这一不等式不再成立，Dijkstra 算法的正确性也将无法得到保证。

### ✓ 复杂度分析：

朴素的实现方法为每次 2 操作执行完毕后，直接在  $T$  集合中暴力搜索寻找最短路长度最小的节点。2 操作的总时间复杂度为  $O(m)$ ，1 操作总时间复杂度为  $O(n^2)$ ，全过程时间复杂度为  $O(n^2 + m) = O(n^2)$ 。

我们实际上可以通过堆等数据结构对该过程进行优化，这里不作详细说明。

示例代码如下：

```
int n, m; // 顶点数、边数
int graph[MAXN][MAXN]; // 邻接矩阵表示图
int dis[MAXN]; // 源点到每个点的最短距离
int visited[MAXN]; // 是否已经确定最短路径

void dijkstra(int start) {
    for (int i = 0; i < n; ++i) {
        dis[i] = INF;
        visited[i] = 0;
    }
    dis[start] = 0;

    for (int i = 0; i < n; ++i) {
        int u = -1, min_dis = INF;
        // 在未访问的点中选择距离最小的点
        for (int j = 0; j < n; ++j) {
            if (!visited[j] && dis[j] < min_dis) {
                min_dis = dis[j];
                u = j;
            }
        }
        if (u == -1) break; // 所有可达点已处理
        visited[u] = 1;

        // 对 u 的邻接点进行松弛
        for (int v = 0; v < n; ++v) {
            if (graph[u][v] != INF && dis[v] > dis[u] + graph[u][v]) {
                dis[v] = dis[u] + graph[u][v];
            }
        }
    }
}

int main() {
    scanf("%d %d", &n, &m); // 输入点数和边数
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            graph[i][j] = (i == j ? 0 : INF);

    for (int i = 0; i < m; ++i) {
        int u, v, w;
```

```

scanf("%d %d %d", &u, &v, &w);
if (w < graph[u][v]) // 重边取最小
    graph[u][v] = w;
}

int start;
scanf("%d", &start); // 源点编号

dijkstra(start);

for (int i = 0; i < n; ++i) {
    if (dis[i] == INF)
        printf("从 %d 到 %d 的最短路不存在\n", start, i);
    else
        printf("从 %d 到 %d 的最短路为 %d\n", start, i, dis[i]);
}

return 0;
}

```

## Network Flow

### Intro

网络指的是一个有向图  $G = (V, E)$  满足：

- 每个有向边  $e$  有一个非负容量  $c_e$
- 有一个源结点  $s$  没有入边
- 有一个目标点  $t$  没有出边
- 假设对于每个结点，都有一条经过该结点的从  $s$  到  $t$  的路径

有向图  $G = (V, E)$  中的  $s \rightarrow t$  流是一个实值函数  $f: V \times V \rightarrow R^+$ ，且满足

- 容量条件：对每个边  $e$ ， $0 \leq f(e) \leq c_e$
- 保存条件 (流量守恒)：除源汇点外，任意节点  $u$  的净流量为 0。其中，我们定义  $u$  的净流量为  $\sum_{x \in V} f(u, x) - \sum_{x \in V} f(x, u)$ ，也就是说  $\sum_{x \in V} f(u, x) = \sum_{x \in V} f(x, u)$ 。
- 源和汇满足： $\sum_{x \in V} f(s, x) = \sum_{x \in V} f(x, t)$ 。

给定图  $G = (V, E)$  中  $s \rightarrow t$  流  $f$  和任意结点集合  $S$

- 流入量： $f^{in}(S) = \sum_{e \text{ in } S} f(e)$
- 流出量： $f^{out}(S) = \sum_{e \text{ out } S} f(e)$

- 满足:  $f^{in}(S) = f^{out}(S)$

令  $G = (V, E)$  是一个有源汇点的网络, 我们希望在  $G$  上指定合适的流  $f$ , 以最大化整个网络的流量

$|f|$  (即  $\sum_{x \in V} f(s, x) - \sum_{x \in V} f(x, s)$ ), 这一问题被称作最大流问题 (Maximum flow problem)。

## Ford-Fulkerson

关于该算法, [OI-Wiki](#) 有着极为详尽的描述、推导和证明, 以下仅为课件部分。

给定网络  $G$  及  $G$  上的流  $f$ , 我们做如下定义:

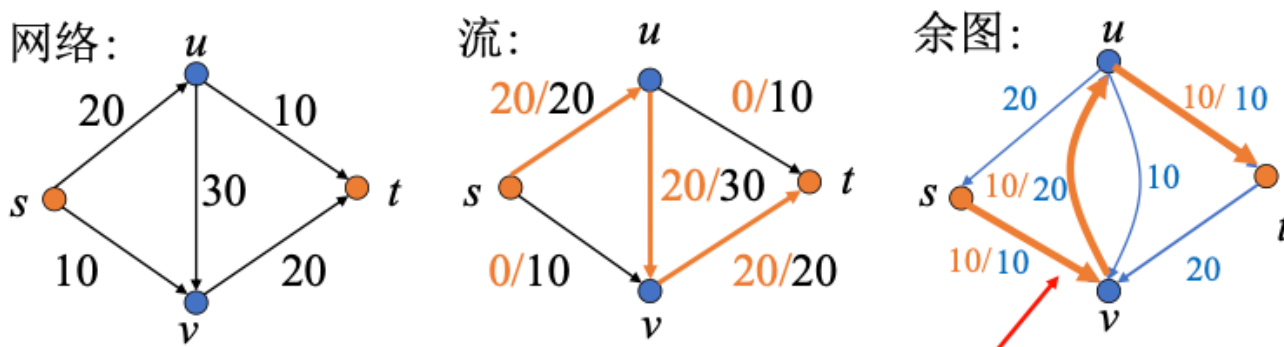
- 对于边  $(u, v)$ , 我们将其容量与流量之差称为剩余容量 (Residual Capacity)  $c_f(u, v)$ , 即

$$c_f(u, v) = c(u, v) - f(u, v)。$$

- 我们将  $G$  中所有结点和剩余容量大于 0 的边构成的子图称为残量网络 (Residual Network)  $G_f$ , 即

$$G_f = (V, E_f), \text{ 其中 } E_f = \{(u, v) | c_f(u, v) > 0\}。$$

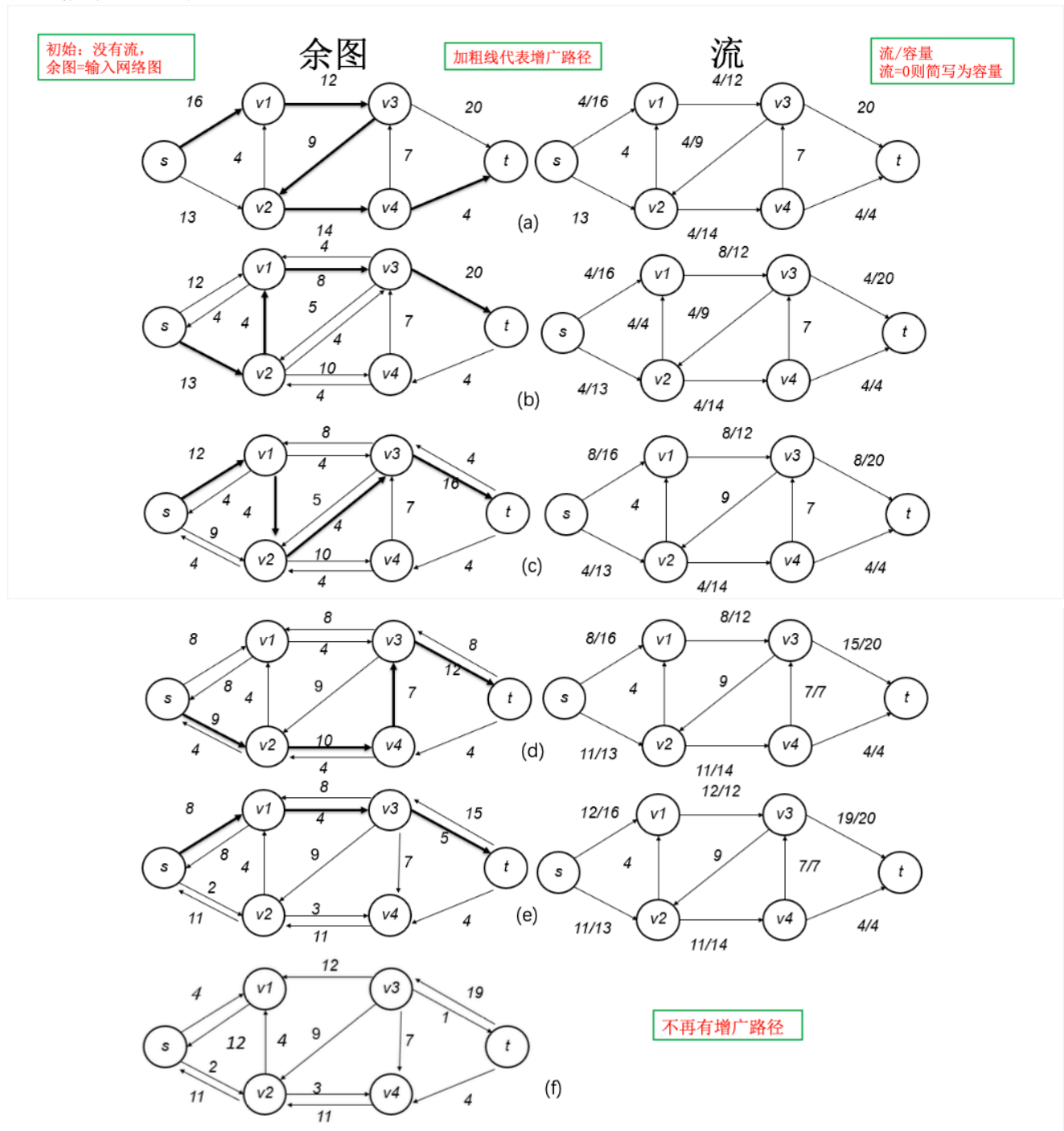
- 在算法过程中, 对于每条边  $(u, v)$ , 我们都新建一条反向边  $(v, u)$ 。我们约定  $f(u, v) = -f(v, u)$ , 因此每次  $f(u, v)$  增加时  $f(v, u)$  都应当减少同等的量。
- 增广路径: 找到残量网络  $G_f$  中的一条流, 该流通过一条没有重复结点的路径  $p$ , 并且值和该路径  $p$  上的最小容量相等。



📄 算法思想:

- 初始, 对于所有的边  $e$  初始化流  $f(e) = 0$ ;
- 每一次迭代中, 增加图中的流值, 同时更新残差网络。方法: 找到残量网络中的一条新流, 该流通过一条没有重复结点的路径, 并且值和该路径上的最小容量相等 (增广路径), 该值为沿着增广路径能够为每条边增加的流量的最大值;
- 不断迭代, 直到不存在增广路径为止, 即流值已经最大化。

极为抽象的过程如下：



正确性证明：

课件上讲的基本是废话。[OI-Wiki](https://oi-wiki.org/flow/) 有详细的证明。

示例代码如下：

```
int n, m, s, t;
int capacity[MAXN][MAXN]; // 容量图
int flow[MAXN][MAXN];      // 当前流量
int visited[MAXN];         // DFS访问标记

int min(int a, int b) {
    return a < b ? a : b;
}
```

```

// DFS 寻找增广路径，返回瓶颈流量
int dfs(int u, int min_flow) {
    if (u == t) return min_flow;
    visited[u] = 1;
    for (int v = 0; v < n; ++v) {
        int residual = capacity[u][v] - flow[u][v];
        if (!visited[v] && residual > 0) {
            int bottleneck = dfs(v, min(min_flow, residual));
            if (bottleneck > 0) {
                flow[u][v] += bottleneck;
                flow[v][u] -= bottleneck; // 反向边更新
                return bottleneck;
            }
        }
    }
    return 0;
}

// 主过程
int ford_fulkerson() {
    int max_flow = 0;
    while (1) {
        memset(visited, 0, sizeof(visited));
        int augment = dfs(s, INF);
        if (augment == 0) break; // 没有增广路径
        max_flow += augment;
    }
    return max_flow;
}

int main() {
    scanf("%d %d %d %d", &n, &m, &s, &t); // 点数，边数，源点，汇点
    memset(capacity, 0, sizeof(capacity));
    memset(flow, 0, sizeof(flow));

    for (int i = 0; i < m; ++i) {
        int u, v, cap;
        scanf("%d %d %d", &u, &v, &cap);
        capacity[u][v] += cap; // 累加重边
    }

    int result = ford_fulkerson();
    printf("最大流为: %d\n", result);
    return 0;
}

```